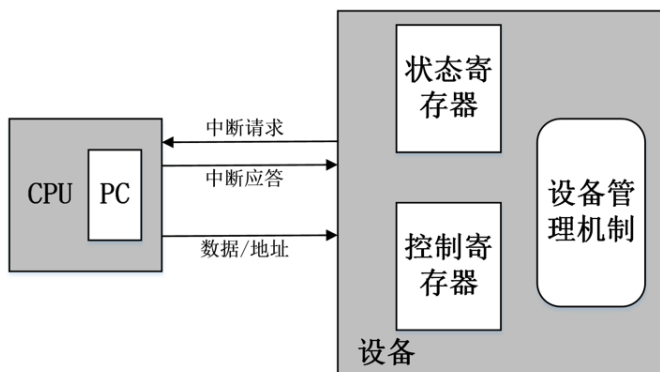


第 06 讲 嵌入式处理器中断处理

一、 嵌入式处理器中断

➤ 中断

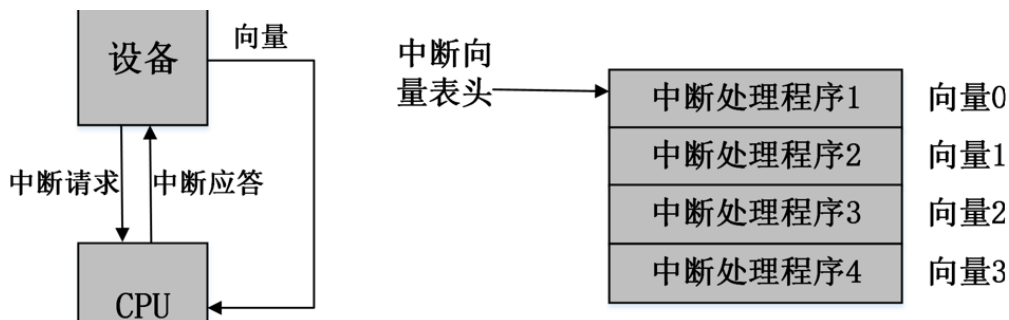


- 中断请求?
如何处理
- 中断应答?
如何实现
- 数据地址去
拿数据, 如
何实现?

- ✓ CPU 通过在执行每条指令前检查中断请求线上是否有中断
- ✓ 如果有中断发生, CPU 就不去读取 PC 所指的指令, 而是让 PC 指向一个预先设置好的位置
- ✓ 中断处理程序开始位置通常是一个指针, 而不是一个固定的位置
- ✓ 在此时需要对前台程序的位置进行记录, 因此需要用到栈

➤ 中断向量表

中断向量线路从设备连到 CPU, 在设备请求被应答后, 它通过这条线路将中断向量发到 CPU, 然后 CPU 将向量号作为存储在内存中的中断向量表的索引。中断向量表中由向量号指定的位置给出了中断程序地址。

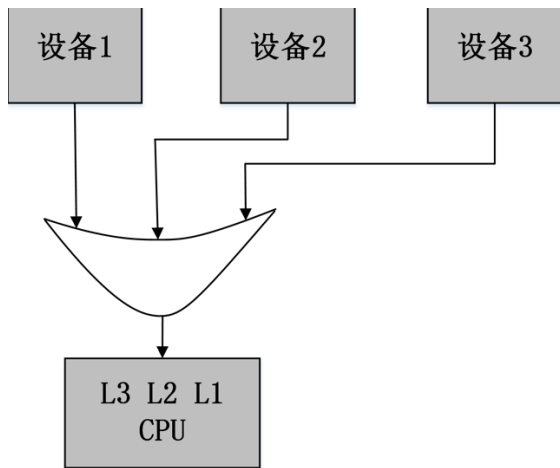


大多数系统不止一个 I/O 设备, 因此一个实用的中断系统需要多个中断请求线路
中断优先级可以使得 CPU 容易进行区别, 配置
中断向量能让中断设备指定中断程序

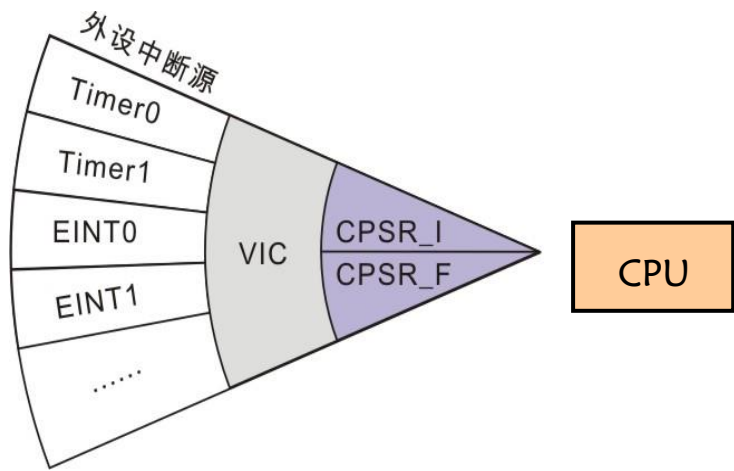
优先级机制使得高优先级中断时候, 不发生低优先级中断, 该决策过程称之为屏蔽。
最高优先级中断一般称为不可屏蔽中断 NMI, 不能被关掉, 比如电源故障中断

➤ 向量中断控制器

概述: ARM 内核具有两个中断输入, 分别为 IRQ 中断和 FIQ 中断。4 个向量中断控制器 (VIC) 负责管理芯片的中断源, 最多可以管理 93 个中断输入请求。



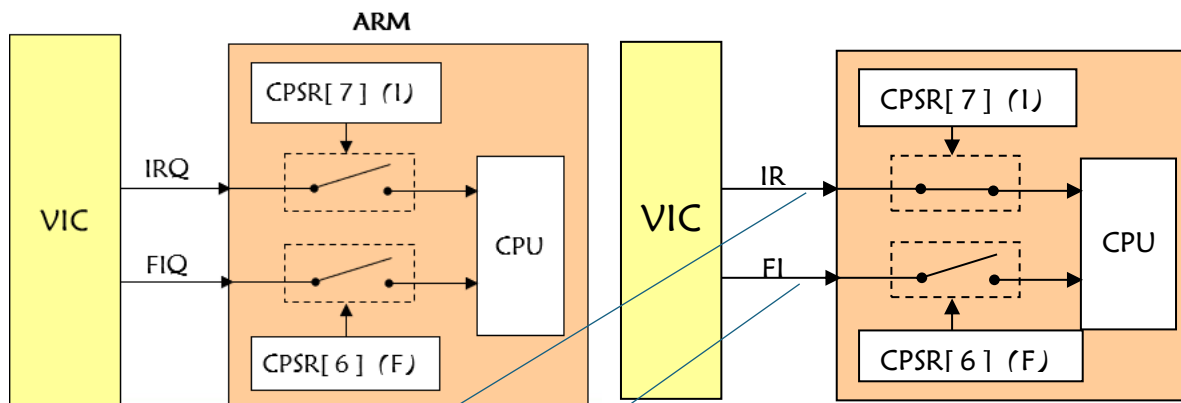
使用轮询让几个设备共享一个中断



向量中断控制器

• 程序状态寄存器 CPSR 与 VIC 的关系

ARM 内核通过 CPSR 来监视和控制内部的操作，CPSR 中的“I”位和“F”位分别用来控制 IRQ 模式和 FIQ 模式的使能。



当 I = 0 时，使能 IRQ 中断
当 I = 1 时，禁止 IRQ 中断

当 F = 0 时，使能 FIQ 中断
当 F = 1 时，禁止 FIQ 中断

• 中断分类

中断输入请求可以在 VIC 中被设置为以下二类：

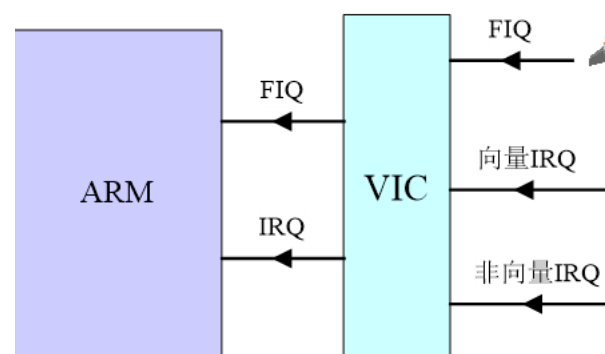
- **FIQ 中断**：具有最高优先级；
- **IRQ 中断**：具有一般优先级；

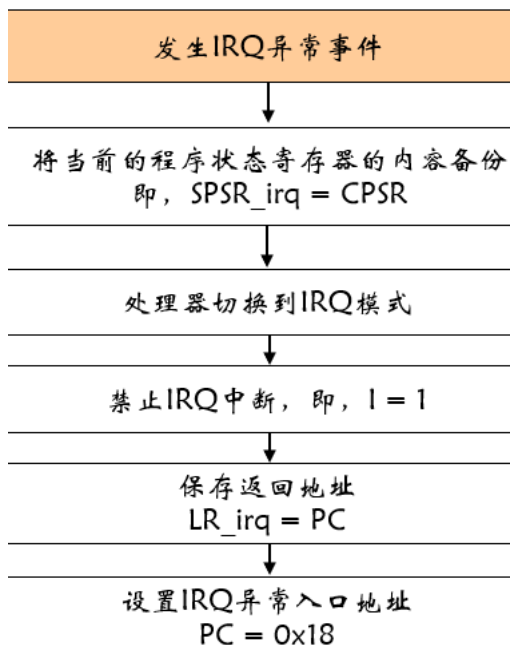
向量 IRQ-----

由硬件提供中断服务程序入口地址，
不同的中断有不同的入口地址，
用于 RESET、NMI、异常处理；

非向量 IRQ-----

由软件提供中断服务程序入口地址，
只有一个入口地址，
进去了在判断中断标志来识别具体是哪个中断





寄存器描述 - 参数设置寄存器

向量地址0寄存器 VICVectAddr0		向量地址31寄存器 VICVectAddr31	向量地址寄存器 VICVectAddr
--------------------------	-------	----------------------------	------------------------

名称	描述	复位值	地址
VICVectAddr0 ~ VICVectAddr31	向量地址0寄存器 ~ 向量地址31寄存器	0	0xF200_0100 ~ 0xF200_017C
VIC0Addr	中断地址寄存器	0	0xF200_0F00

寄存器描述 - 控制寄存器

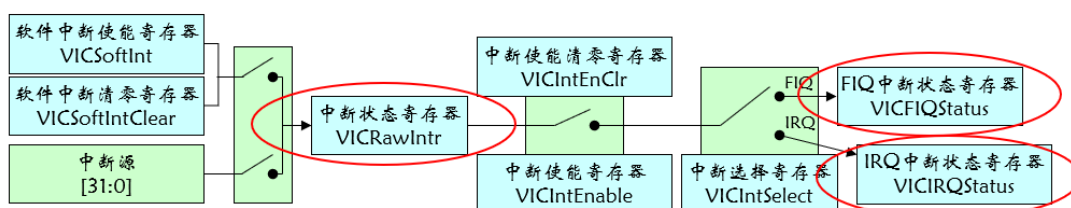
控制寄存器功能描述

控制寄存器	功能描述
VICIntEnable	使能 (禁止) 中断源产生中断
VICIntEnClr	
VICIntSelect	中断类型选择: FIQ & IRQ
VIC0VectADDR	中断地址设置

名称	描述	访问	复位值	地址
VIC0IntSelect	中断选择寄存器 将32个中断请求的每个中断分配为FIQ或IRQ	R/ W	0	0xF200_000C
VIC0IntEnable	中断使能寄存器 控制32个中断请求 (包括软件中断) 的使能	R/ W	0	0xF200_0010
VIC0IntEnClr	中断使能清零寄存器 将中断使能寄存器中的一个或多个位清零	W	0	0xF200_0014



寄存器描述 - 状态寄存器



名称	描述	访问	复位值	地址
VICIRQStatus	IRQ状态寄存器 该寄存器读出定义为IRQ并使能的中断的状态	RO	0	0xFFFF F000
VICFIQStatus	FIQ状态寄存器 该寄存器读出定义为FIQ并使能的中断的状态	RO	0	0xFFFF F004
VICRawIntr	所有中断的状态寄存器 该寄存器读出32个中断请求/软件中断的状态, 不管中断是否使能或分类	RO	0	0xFFFF F008

注意: 读取VICRawIntr寄存器将得到所有32个中断请求和软件中断的状态, 它不管中断是否使能或分类。

• IRQ中断处理

硬件处理

①	SPSR_irq = CPSR
②	CPSR = nzcvqIf ₁ t_irq
③	VICVectAddr = VICVectAddrn
④	LR_irq = PC
⑤	PC = 0x18

软件处理

①	获取中断服务程序地址
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR

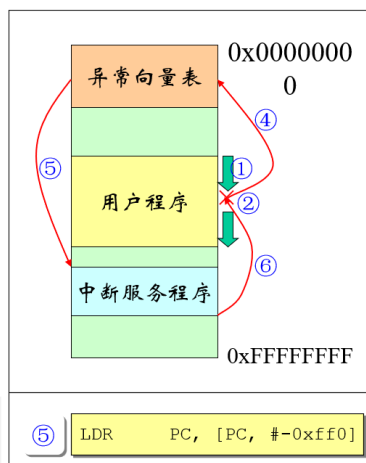
①	PC ← [VICVectAddr]
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR

①	LDR PC, [PC, #-0xff0]
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR

SUBS PC, LR, #4

• 图示IRQ中断的发生过程

- 1.正在执行用户程序;
- 2.外部中断0发生中断;
- 3.VIC硬件将中断服务程序地址装入VICVectAddr寄存器;
- 4.程序跳转至异常向量表中IRO入口0x0018处;
- 5.执行指令跳转至VICVectAddr寄存器中的中断服务地址;
- 6.中断服务程序执行完毕,返回被中断的用户程序继续执行被中断的代码。



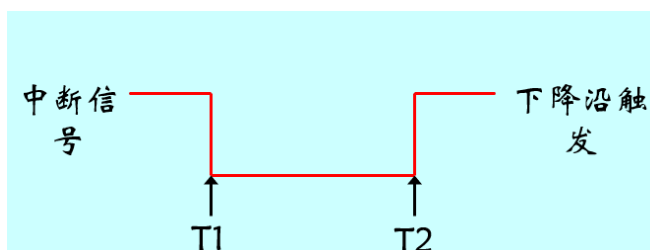
➤ 外部中断输入

• 边沿触发中断

下降沿触发类型中断的请求和清除时序。

T1时刻, 中断信号由下降沿产生, 中断控制器向CPU发出中断请求。

T2时刻, CPU执行完成中断控制器的中断服务程序, 清除中断, 中断信号回复到高电平。



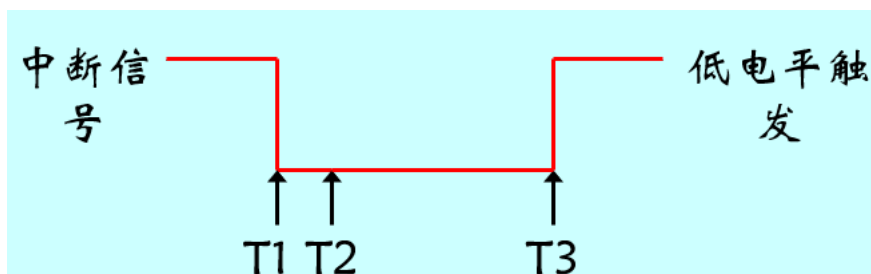
• 电平触发中断

低电平触发类型中断的请求和清除时序。

T1时刻, 中断信号开始由高电平转为低电平。

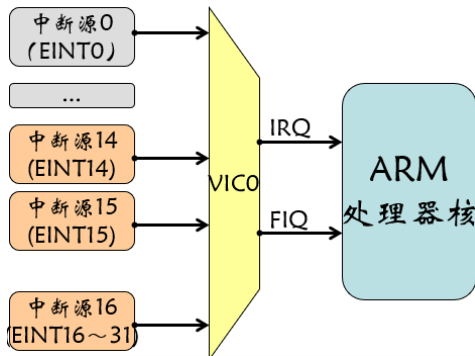
T2时刻, 中断控制器确认中断信号是低电平后, 将向CPU发出中断请求。

T3时刻, CPU执行完成中断控制器的中断服务程序, 清除中断, 中断信号回复到高电平。



- 外部中断源

所以的外部中断源都由 VIC0 管理，其中外部中断含有 16 个独立的中断号和一个共用的中断号。



二、 嵌入式中断驱动程序

➤ 中断

一个驱动程序只需要为它自己设备的中断注册一个处理函数，系统会在适当的时候调用它。

中断号是非常珍贵的系统资源，模块要在使用前先请求一个中断通道，在使用后释放它。

➤ 申请中断号

```
#include <linux/interrupt.h>
```

```
typedef irqreturn_t (*irq_handler_t)(int, void *);
```

```
int request_irq(unsigned int irq,  
               irq_handler_t handler,  
               unsigned long flags,  
               const char *name,  
               void *dev);
```

```
void free_irq(unsigned int irq, void *dev);
```

参数说明：

unsigned int irq

要请求的中断号

irqreturn_t (*handler)(int, void *)

中断处理函数指针

unsigned long flags

与中断有关的位掩码

const char *name

在 /proc/interrupts 显示中断拥有者的字符串

void *dev

request_irq 函数传递给 handler 中断函数的 void *参数

➤ 中断共享

由于中断信号线非常珍贵，所以共享中断是难以避免的

请求中断时，**必须指定 flags 参数中的 SA_SHIRQ 位。**

➤ 申请、释放中断的时机

调用 request_irq 的正确位置应该是在设备**第一次打开、硬件被告知产生中断之前**。一般是在设备注册时候进行调用，比如 register_chardev 函数之后

必须判断中断注册 request_irq 的返回值

调用 free_irq 的位置是在最后一次关闭设备、硬件被告知不要再中断处理器之后。

➤ 中断处理程序的实现

```
irqreturn_t interrupt_handler(int irq, void *dev){
```

中断处理函数是一个普通的函数，没有什么特别的地方。但由于是在中断期间运行，不与任何进程上下文相关，因而受到一些限制：

- ✓ 不能向用户空间发送和接收数据
- ✓ 不能做任何可能发生睡眠的操作等

为了实时性考虑，不能在中断中处理过多任务，耗时不能太长

➤ 中断处理程序的局限性

- 负责对硬件做出迅速响应并完成时间要求很严格的操作
 - 异步方式执行，可能会打断其他重要代码的执行
 - 中断处理程序执行过程中
 - 最好情形下，同级中断被屏蔽
 - 最坏情形下，当前处理器上所有其他中断都会被屏蔽
- 中断处理程序不在进程上下文中运行，不能被阻塞

➤ 中断处理流程

- 上半部：中断处理程序
 - 简单快速，执行时禁止部分或全部中断
- 下半部
 - 执行与中断处理密切相关但中断处理程序本身不执行的工作
 - 执行期间可以响应中断

➤ 上半部与下半部的划分

没有严格的划分规则

- 如果任务对时间非常敏感，放在中断处理程序中执行
- 如果任务与硬件相关，放在中断处理程序中执行
- 如果任务需确保不被其他中断（同级中断）打断，放在中断处理程序中执行
- 其他任务，考虑放置在下半部执行

➤ 下半部实现机制的演化过程

tasklet 实现在软中断之上
tasklet 和软中断也称为可延迟函数

下半部机制	状态
BH	在2.5中移除
任务队列	在2.5中移除
软中断	从2.3开始引入
Tasklet	从2.3开始引入
工作队列	从2.3开始引入

➤ 可延迟函数的基本操作

- 初始化
 - 定义一个新的可延迟函数，该操作通常在内核初始化或加载模块时进行
- 激活

标记一个可延迟函数为“挂起”（在可延迟函数的下一轮调度中执行）
激活可以在任何时候进行（即使正在处理中断）

- 屏蔽
有选择地屏蔽一个函数，使其即使被激活，内核也不执行
- 执行
执行一个挂起的可延迟函数及同类型其他所有挂起的可延迟函数
执行是在特定的时间上进行的

三、 软中断

➤ 软中断概念

软中断的一种典型应用就是所谓的“下半部”（bottom half），它的得名来自于将硬件中断处理分离成“上半部”和“下半部”两个阶段的机制：上半部在屏蔽中断的上下文中运行，用于完成关键性的处理动作；而下半部通常比较耗时的，由系统自行安排运行时机，不在中断服务上下文中执行。

一组静态定义的下半部接口

有 32 个，必须在编译阶段静态注册

一个软中断不会抢占另一个软中断

软中断处理过程中允许响应中断，但自己不能休眠

在一个处理程序运行的时候，当前处理器上的软中断被禁止

tasklet	优 先 级	软中断描述
HI_SOFTIRQ	0	优先级高的tasklet
TIMER_SOFTIRQ	1	定时器的下半部
NET_TX_SOFTIRQ	2	发送网络数据包
NET_RX_SOFTIRQ	3	接收网络数据包
SCSI_SOFTIRQ	4	SCSI的下半部
TASKLET_SOFTIRQ	5	tasklet

➤ 软中断的索引分配

编译期间，在<linux/interrupt.h>中定义一个枚举类型来静态地声明软中断
索引号小的软中断在索引号大的软中断之前执行——索引号越小优先级越高
添加新的索引项必须考虑优先级

- ❑ 一般将 HI_SOFTIRQ 作第一项，TASK_SOFTIRQ 做最后一项

➤ 软中断相关结构定义

➤ 软中断结构定义：

softirq_action[linux/interrupt.h]

```
struct softirq_action{  
    //待 执 行 参 数  
    void (*action)(struct softirq_action *);  
    void *data; /*传给函数的参数*/  
};
```

➤ 软中断结构数组[kernel/softirq.h]

- ❑ 软中断数目无法改变
- ❑ 当前内核仅使用到6个

```
static struct softirq_action softirq_vec[32]
```

➤ 处理软中断

➤ 软中断的初始化

```
void open_softirq(int nr, void (*action)(struct softirq_action*),  
void *data){  
    softirq_vec[nr].data = data;  
    softirq_vec[nr].action = action;  
}
```

❑ 举例

```
open_softirq(NET_TX_SOFTIRQ, net_tx_action, NULL);  
open_softirq(NET_RX_SOFTIRQ, net_rx_action, NULL);
```

➤ 软中断的激活

```
extern void raise_softirq(unsigned int nr);
```

➤ 软中断的执行

```
void __do_softirq(void)
```

➤ 函数原型

```
extern void raise_softirq(unsigned int nr);
```

➤ 软中断的激活

➤ 处理流程

- ❑ 保存flags寄存器IF标志的状态并禁止本地cpu上的中断
- ❑ 把软中断设置为挂起状态，使其在下次调用do_softirq()时能够运行
- ❑ 调用in_interrupt()
 - 如果产生为1的值（说明已经在中断上下文中调用了raise_softirq()或当前禁用了软中断），直接恢复IF标志的状态值
 - 否则，在需要的时候调用wakeup_softirqd()唤醒本地cpu的ksoftirqd内核线程并恢复IF标志的状态值

➤ raise_softirq()代码结构

```
void raise_softirq(unsigned int nr){
    unsigned long flags;
    local_irq_save(flags);
    raise_softirq_irqoff(nr);
    local_irq_restore(flags);
}

inline void raise_softirq_irqoff(unsigned int nr){
    __raise_softirq_irqoff(nr);
    if (!in_interrupt())
        wakeup_softirqd();
}

#define __raise_softirq_irqoff(nr) do { or_softirq_pending(1UL << (nr)); } while (0)

static inline void wakeup_softirqd(void){
    /* Interrupts are disabled: no need to stop preemption */
    struct task_struct *tsk = __get_cpu_var(ksoftirqd);
    if (tsk && tsk->state != TASK_RUNNING)
        wake_up_process(tsk);
}
```

➤ do_softirq()的操作流程

➤ 在local_softirq_pending不为0时执行

- ❑ 如果in_interrupt()产生值为1，则函数返回
- ❑ 保存flags寄存器IF标志的状态并禁止本地cpu上的中断
- ❑ 调用__do_softirq()

```
asmlinkage void do_softirq(void){
    __u32 pending;
    unsigned long flags;
    if (in_interrupt())
        return;
    local_irq_save(flags);
    pending = local_softirq_pending();
    if (pending)
        do_softirq();
    local_irq_restore(flags);
}
```

➤ __do_softirq()的操作流程

读取本地 CPU 的软中断掩码并执行与每个设置位对应的可延迟函数

存在问题：

由于在执行一个软中断函数期间，可能会出现新挂起的软中断，为保证可延迟函数的低延迟性，该函数一直运行到执行完所有挂起的软中断

但这种机制可能会迫使__do_softirq()运行很长时间，影响用户态进程的执行

解决途径：

固定每次的循环次数，然后就返回

其余挂起的软中断在内核线程 ksoftirqd 中执行

➤ __do_softirq()代码

```
#define MAX_SOFTIRQ_RESTART 10
asmlinkage void __do_softirq(void) {
    struct softirq_action *h;
    __u32 pending;
    int max_restart = MAX_SOFTIRQ_RESTART;
    int cpu;
    pending = local_softirq_pending();
    account_system_vtime(current);
    // 屏蔽其他软中断，使每个CPU上只能同时运行一个软中断
    __local_bh_disable((unsigned long) __builtin_return_address(0));
    trace_softirq_enter();
    // 针对 SMP 得到当前正在处理的 CPU
    cpu = smp_processor_id();
restart:
    set_softirq_pending(0);
    // 此时才开中断运行，以前运行状态一直是关中断运行
    // 这时当前处理软中断才可能被硬件中断抢占。
    local_irq_enable();
    // 以下代码可被硬件中断抢占，但无法注册相应软中断
    h = softirq_vec;
    do {
        if (pending & 1) {
            h->action(h);
            rcu_bh_qsctr_inc(cpu);
        }
        h++;
        pending >>= 1;
    } while (pending);
    // 以下代码执行过程中硬件中断无法抢占。
    local_irq_disable();

    // 因为刚才开中断执行过程中可能多次被硬件中断抢占，每抢占一次就有可
    // 能注册一个软中断，所以要再重新取一次所有的软中断。
    pending = local_softirq_pending();
    if (pending && --max_restart)
        goto restart;
    // 如果以上步骤重复了 10 次后还有 pending 的软中断的话，调用 ksoftirqd
    if (pending)
        wakeup_softirqd();
    trace_softirq_exit();
    account_system_vtime(current);
    __local_bh_enable();
}
```

➤ ksoftirqd 内核线程

被唤醒时，检查 local_softirq_pending 中的软中断并在必要时调用 do_softirq()

如果没有挂起的软中断，把当前进程状态设置为休眠，随后若当前进程需要，就调用 cond_resched() 函数实现进程切换

➤ Ksoftirqd 代码

```

static int ksoftirqd(void * __bind_cpu) {
    set_user_nice(current, 19);
    // 设置当前进程不允许被挂启
    current->flags |= PF_NOFREEZE;
    set_current_state(TASK_INTERRUPTIBLE);
    while (!kthread_should_stop()) {
        //若可处理，在此处理期间内禁止当前进程被抢占
        preempt_disable();
        // 首先判断系统当前没有需要处理的 pending 状态的软中断
        if (!local_softirq_pending()) {
            // 没有的话在主动放弃 CPU 前要先允许抢占
            preempt_enable_no_resched();
            //将当前进程放入睡眠队列，并切换新的进程执行
            schedule();
            // 再次被调度时，将从调用这个函数的下一条语句开始执行
            preempt_disable();
        }
        __set_current_state(TASK_RUNNING);

        while (local_softirq_pending()) {
            if (cpu_is_offline((long) __bind_cpu))
                // 如果当前被关联的 CPU 无法继续处理则跳转
                // 到 wait_to_die 标记出，等待结束并退出。
                goto wait_to_die;
            do_softirq();
            // 允许当前进程被抢占。
            preempt_enable_no_resched();
            // 该函数有可能间接的调用 schedule() 来切换当前进程
            cond_resched();
            preempt_disable();
        }
        preempt_enable();
        set_current_state(TASK_INTERRUPTIBLE);
    }
    // 如果将会停止则设置当前进程为运行状态后直接返回。
    __set_current_state(TASK_RUNNING);
    return 0;

wait_to_die:
    preempt_enable();
    set_current_state(TASK_INTERRUPTIBLE);
    while (!kthread_should_stop()) {
        schedule();
        set_current_state(TASK_INTERRUPTIBLE);
    }
    __set_current_state(TASK_RUNNING);
    return 0;
}

```

➤ 使用软中断

软中断预留给系统中对时间要求最严格以及最重要的下半部使用

直接使用软中断的子系统

网络

SCSI

间接使用软中断的子系统

内核定时器

Tasklet

四、 Tasklet 机制

➤ tasklet 概念

- 基于软中断实现的灵活性强、动态创建的下半部机制

类型相同的 tasklet 不能同时执行

两种不同类型的 tasklet 可以在不同处理器上同时执行

tasklet 由 HI_SOFTIRQ 及 TASKLET_SOFTIRQ 两类软中断代表组成，前者的优先级高于后者

- tasklet 与软中断的权衡

通常应该使用 tasklet

软中断适用于执行频率很高且连续性要求很高的情况

➤ tasklet 结构体

➤ tasklet_struct[linux/interrupt.h], 每个结构体单独代表一个tasklet

```
struct tasklet_struct{
    struct tasklet_struct *next; /*下一个 tasklet*/
    unsigned long state; /*tasklet的状态*/
    atomic_t count; /*引用计数器*/
    void (*func)(unsigned long); /*tasklet处理函数*/
    unsigned long data; /*处理函数所需参数*/
};
```

□ state

- 0: 初始状态
- TASKLET_STAT_SCHED: 已被调度，等待运行
- TASKLET_STAT_RUN: 正在运行该状态，只有在多处理器系统上才会作为优化来使用

□ count

- 非0: 禁止tasklet
- 等于0: 启用tasklet

➤ tasklet 的初始化

➤ 静态创建

- 使用<linux/interrupt.h>中定义的两个宏之一

```
#define DECLARE_TASKLET(name, func, data) \
struct tasklet_struct name = { NULL, 0, ATOMIC_INIT(0), func, data }

#define DECLARE_TASKLET_DISABLED(name, func, data) \
struct tasklet_struct name = { NULL, 0, ATOMIC_INIT(1), func, data }
```

➤ 动态创建

- 将一个间接引用赋给一个动态创建的tasklet_struct结构

```
void tasklet_init(struct tasklet_struct *t,
    void (*func)(unsigned long), unsigned long data){
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0);
    t->func = func;
    t->data = data;
}
```

➤ tasklet 处理程序的编写

➤ 处理函数原型形式

```
void tasklet_handler(unsigned long data)
```

➤ 函数定义说明

❑ 不能在处理程序内部使用信号量或其他阻塞式函数

❑ 由于tasklet运行时允许响应中断，如果tasklet与中断处理程序之间共享了某些数据，必须做好预防工作（锁机制）

➤ 禁止指定 tasklet

➤ 函数原型

❑ tasklet_disable()

• 如果该tasklet正在执行，将等待执行完毕后再返回

```
static inline void tasklet_disable(struct tasklet_struct *t){
    tasklet_disable_nosync(t);
    tasklet_unlock_wait(t);
    smp_mb();
}
```

❑ tasklet_disable_nosync()

• 无须在返回前等待tasklet执行完毕

```
static inline void tasklet_disable_nosync(struct tasklet_struct *t){
    atomic_inc(&t->count);
    smp_mb__after_atomic_inc();
}
```

➤ tasklet 的删除

➤ 从挂起的队列中去掉一个tasklet

```
void tasklet_kill(struct tasklet_struct *t){
    if (in_interrupt())
        printk("Attempt to kill tasklet from interrupt\n");
    while (test_and_set_bit(TASKLET_STATE_SCHED, &t->state)) {
        do
            yield();
        while (test_bit(TASKLET_STATE_SCHED, &t->state));
    }
    tasklet_unlock_wait(t);
    clear_bit(TASKLET_STATE_SCHED, &t->state);
}

void tasklet_kill_immediate(struct tasklet_struct *t, unsigned int cpu){
    struct tasklet_struct **i;
    BUG_ON(cpu_online(cpu));
    BUG_ON(test_bit(TASKLET_STATE_RUN, &t->state));
    if (!test_bit(TASKLET_STATE_SCHED, &t->state))
        return;
    /* CPU is dead, so no lock needed. */
    for (i = &per_cpu(tasklet_vec, cpu).head; *i; i = &(*i)->next) {
        if (*i == t) {
            *i = t->next;
            /* If this was the tail element, move the tail ptr */
            if (*i == NULL)
                per_cpu(tasklet_vec, cpu).tail = i;
            return;
        }
    }
    BUG();
}
```

➤ tasklet 的调度

➤ 已挂起的tasklet存放在两个单处理数据结构，两结构都是由tasklet_struct结构体组成的链表

❑ task_vec: 普通tasklet

❑ task_hi_vec: 高优先级tasklet

➤ 调度函数

❑ 函数原型

• tasklet_schedule()

• tasklet_hi_schedule()

❑ 说明

• 均接收一个指向tasklet_struct结构的指针作为参数

• 两函数区别在于分别使用TASKLET_SOFTIRQ和HI_SOFTIRQ

➤ tasklet 调度的流程

■ 本质:激活相应软中断的挂起状态

- ◆ 检查 TASKLET_STATE_SCHED 标志, 若设置则返回
- ◆ 调用 local_irq_save 保存 IF 标志的状态并禁用本地中断
- ◆ 在 tasklet_vec[n]或 tasklet_hi_vec[n]指向的链表其起始处增加 tasklet 描述符 (n 代表 CPU 的逻辑号)
- ◆ 调用 raise_softirq_irqoff()或 raise_hi_irqoff()激活相应类型中断
- ◆ 调用 local_irq_restore 恢复 IF 标志的状态并禁用本地中断

➤ tasklet 软中断处理函数

■ 处理函数: tasklet_action()/tasklet_hi_action()

- ◆ 禁止中断 (此时无须保存其状态), 并为当前 CPU 检索 tasklet_vec/task_hi_vec
- ◆ 将当前处理器上的该链表设置为 NULL, 达到清空效果
- ◆ 打开中断
- ◆ 循环遍历获得链表上的每个待处理 tasklet
 - 如果是多处理系统, 通过检查 TASKLET_STATE_RUN 状态标志来判断这个 tasklet 是否正在其他处理器上运行
 - 若在, 跳到下一个待处理 tasklet
 - 如果当前 tasklet 没有执行, 将其状态标志为 TASKLET_STATE_RUN
 - 检查 count 是否为 0, 确保 tasklet 没有被禁止
 - 若禁止, 则跳到下一个挂起的 tasklet
 - 执行 tasklet 对应的处理程序, 执行完毕后, 清除 state 域的 TASKLET_STATE_RUN 标志
- ◆ 重复执行下一个 tasklet, 直到没有剩余待处理的 tasklet

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/interrupt.h>

static struct tasklet_struct my_tasklet;

static void tasklet_handler (unsigned long data)
{
    printk(KERN_ALERT "tasklet_handler is running.\n"); }

static int __init demo_init(void)
{
    tasklet_init(&my_tasklet, tasklet_handler, 0);
    tasklet_schedule(&my_tasklet);
    printk(KERN_ALERT "demo_init.\n");
    return 0; }

static void __exit demo_exit(void)
{
    tasklet_kill(&my_tasklet);
    printk(KERN_ALERT "demo_exit.\n"); }

MODULE_LICENSE("GPL");
module_init(demo_init);
module_exit(demo_exit);
```

➤ tasklet 机制小结

- 所有的 tasklet 都通过重复运用 HI_SOFTIRQ 和 TASKLET_SOFTIRQ 这两个软中断来实现
 - ◆ 当一个 tasklet 被调度时，内核会唤醒这两个软中断中的一个
 - ◆ 随后该中断会被特定的函数处理，执行所有已调度的 tasklet
 - ◆ 该函数保证同一时刻只有一个给定类型的 tasklet 会被执行，但其他不同类型的 tasklet 可以同时执行

第 07 讲 Tasklet 与工作队列

一、 中断驱动程序

➤ 中断驱动程序

一个中断驱动程序只需要为它自己设备的中断注册一个处理函数，系统会在适当的时候调用它。
中断号是非常珍贵的系统资源，模块要在使用前先请求一个中断通道，在使用后释放它。

剩余具体内容看上一讲

二、 嵌入式中断驱动程序

二、 Tasklet

具体内容看上一讲的 tasklet 部分

三、 工作队列

➤ 工作队列概述

- 将工作推后，交由一个内核线程去执行
 - ◆ 下半部分在进程上下文执行，通过工作队列执行的代码可以占尽进程上下文的所有优势
 - 支持使用大容量内存
 - 支持获取信号量
 - 支持执行阻塞式 I/O 操作
 - ◆ 工作队列可以用内核线程替换，但一般不建议单独建立对应的内核线程
- 工作队列 vs. 软中断/tasklet
 - ◆ 工作队列支持睡眠，软中断/tasklet 不支持
 - ◆ 若需要使用一个可以重新调度的实体来执行下半部分处理，则应该使用工作队列

➤ 工作队列的实现

- 工作队列子系统是一个用于创建内核线程（工作者线程）的接口，通过它创建的进程负责执行由内核其他部分排队到队列里的任务
- 工作队列子系统提供一个默认的工作者线程（events/n）来处理需推后的工作
 - ◆ 其中 n 是处理器的编号，每个处理器对应一个线程
 - ◆ 工作队列最基本的形式
 - 将需要推后执行的任务交给特定的通用线程
- 工作队列可以让驱动程序创建一个专门的工作者线程来处理需要推后的工作
 - ◆ 主要适用于处理器密集型和性能要求严格的任务

➤ 结构名：work_struct[linux/workqueue.h]

- ❑ 每个处理器的每种类型的队列都对应这样一个链表

➤ 任务数据结构

```
struct work_struct {
    atomic_long_t data;
    struct list_head entry;
    work_func_t func;
#ifdef CONFIG_LOCKDEP
    struct lockdep_map lockdep_map;
#endif
};
```

➤ DECLARE_WORK()

➤ 任务创建方法—静态创建

```
#define DECLARE_WORK(n, f) \
    struct work_struct n = __WORK_INITIALIZER(n, f)

#define DECLARE_DELAYED_WORK(n, f) \
    struct delayed_work n = __DELAYED_WORK_INITIALIZER(n, f)

#define __WORK_INITIALIZER(n, f) { \
    .data = WORK_DATA_INIT(), \
    .entry = { &(n).entry, &(n).entry }, \
    .func = (f), \
    __WORK_INIT_LOCKDEP_MAP(#n, &(n)) \
}

#define __DELAYED_WORK_INITIALIZER(n, f) { \
    .work = __WORK_INITIALIZER((n).work, (f)), \
    .timer = TIMER_INITIALIZER(NULL, 0, 0), \
}
```

➤ 任务创建方法—动态创建

➤ INIT_WORK()

```
#define INIT_WORK(_work, _func) \
do { \
    (_work)->data = (atomic_long_t) WORK_DATA_INIT(); \
    INIT_LIST_HEAD(&(_work)->entry); \
    PREPARE_WORK((_work), (_func)); \
} while (0)

static inline void INIT_LIST_HEAD(struct list_head *list) {
    list->next = list;
    list->prev = list;
}

#define PREPARE_WORK(_work, _func) \
do { \
    (_work)->func = (_func); \
} while (0)
```

➤ 任务中的处理函数

void work_func(struct work_struct *work)

➤ 函数由一个工作者线程执行，会运行在进程上下文中

- ❑ 默认情况下，允许响应中断，并且不持有任何锁
- ❑ 如果需要，函数可以睡眠。
- ❑ 该函数运行在进程上下文中，但不能访问用户空间
 - 因为内核线程在用户空间没有相关的内存映射
 - 通常在系统调用发生时，内核会代表用户空间的进程运行，此时它才能访问用户空间，也只有在此时它才会映射用户空间的内存

➤ 工作队列数据结构

```

struct workqueue_struct {
    struct list_head    pwqs;           /* WR: all pwqs of this wq */
    struct list_head    list;           /* PL: list of all workqueues */

    struct mutex        mutex;           /* protects this wq */
    int                 work_color; /* WQ: current work color */
    int                 flush_color; /* WQ: current flush color */
    atomic_t            nr_pwqs_to_flush; /* flush in progress */
    struct wq_flusher    *first_flusher; /* WQ: first flusher */
    struct list_head    flusher_queue; /* WQ: flush waiters */
    struct list_head    flusher_overflow; /* WQ: flush overflow list */

    struct list_head    maydays; /* MD: pwqs requesting rescue */
    struct worker        *rescuer; /* I: rescue worker */

    int                 nr_drainers; /* WQ: drain in progress */
    int                 saved_max_active; /* WQ: saved pwq max_active */

    struct workqueue_attrs *unbound_attrs; /* WQ: only for unbound wqs */
    struct pool_workqueue *dfl_pwq; /* WQ: only for unbound wqs */

#ifdef CONFIG_SYSFS
    struct wq_device    *wq_dev; /* I: for sysfs interface */
#endif
#ifdef CONFIG_LOCKDEP
    struct lockdep_map    lockdep_map;
#endif
    char                 name[WQ_NAME_LEN]; /* I: workqueue name */

    /* hot fields used during command issue, aligned to cacheline */
    unsigned int         flags __cacheline_aligned; /* WQ: WQ_* flags */
    struct pool_workqueue __percpu *cpu_pwqs; /* I: per-cpu pwqs */
    struct pool_workqueue __rcu *numa_pwq_tbl[]; /* FR: unbound pwqs indexed
} ? end workqueue struct ? ;

```

➤ 工作队列的创建

➤ 函数原型

❑ struct workqueue_struct *

create_workqueue(const char *name);

❑ struct workqueue_struct *

create_singlethread_workqueue(const char *name);

❑ 一个工作队列必须明确的在使用前创建

❑ 每个工作队列有一个或多个专用的进程("内核线程"), 它运行提交给这个队列的函数

❑ create_workqueue 创建一个工作队列, 它有一个专用的线程在系统的每个处理器上.

❑ create_singlethread_workqueue 创建一个工作队列, 它只有单个工作者线程

➤ 提交工作给一个工作队列

■ 函数原型

◆ int queue_work(struct workqueue_struct *queue, struct work_struct *work);

◆ int queue_delayed_work(struct workqueue_struct *queue, struct work_struct *work, unsigned long delay);

◆ 使用 queue_delay_work, 实际的工作没有进行, 直到至少 delay 个 jiffies 已过去

◆ 如果工作被成功加入到工作队列, 返回值是 0

◆ 一个非零返回值意味着这个 work_struct 结构已经在队列中等待, 并且第 2 次没有加入

➤ 工作队列的刷新

■ 函数原型

- ◆ void **flush_workqueue**(struct workqueue_struct *wq);
- ◆ 排入队列的工作会在工作者线程下次被唤醒时执行
- ◆ 有时希望在继续下一步工作之前, 必须保证一些操作已经执行完毕, 可以调用 flush_workqueue:
 - 一直等待, 直到所有对象都被执行后才返回
 - 但不会取消任何延迟执行的工作

➤ 取消延迟执行的工作

■ 函数原型

- ◆ bool **cancel_delayed_work**(struct delayed_work *dwork);
- ◆ 如果这个工作在它开始执行前被取消, 返回值是非零
- ◆ 如果 cancel_delay_work 返回 0, 这个工作可能已经运行在一个不同的处理器, 并且可能仍然在调用 cancel_delayed_work 后再运行
- ◆ 要绝对确保工作函数没有在 cancel_delayed_work 返回 0 后在任何地方运行, **必须跟随这个调用来调用 flush_workqueue 函数**

➤ 工作队列的销毁

■ 函数原型

- ◆ void **destroy_workqueue**(struct workqueue_struct *wq);
- ◆ 当使用完一个工作队列, 可以去销毁它
- ◆ 如果该工作队列上有等待被执行的工作, 则要等到所有工作被执行完毕后再销毁工作队列

➤ 工作的调度

■ 函数原型

- ◆ bool **schedule_work**(struct work_struct *work);
- ◆ 把该工作加入到内核里全局工作队列上
- ◆ bool **schedule_work_on**(int cpu, struct work_struct *work);
- ◆ 把该工作加入到特定 CPU 的工作队列上
- ◆ bool **schedule_delay_work**(struct work_struct *work);
- ◆ bool **schedule_delay_work_on**(int cpu, struct work_struct *work);
- ◆ 延时 delay 个 jiffies 后再把该工作加入到内核全局工作队列或特定 CPU 的工作队列上

➤ 工作者线程

```

static int worker_thread(void * __worker)
{
    struct worker *worker = __worker;
    struct worker_pool *pool = worker->pool;

    /* tell the scheduler that this is a workqueue worker */
    worker->task->flags |= PF_WQ_WORKER;
    woke_up:
    spin_lock_irq(&pool->lock);

    ....

    do {
        struct work_struct *work =
            list_first_entry(&pool->worklist,
                            struct work_struct, entry);

        if (likely(!(*work_data_bits(work) & WORK_STRUCT_LINKED))) {
            /* optimization path, not strictly necessary */
            process_one_work(worker, work);
            if (unlikely(!list_empty(&worker->scheduled)))
                process_scheduled_works(worker);
        } else {
            move_linked_works(work, &worker->scheduled, NULL);
            process_scheduled_works(worker);
        }
    } while (keep_working(pool));

    worker_set_flags(worker, WORKER_PREP, false);
    sleep:
    ... ..|

    worker_enter_idle(worker);
    __set_current_state(TASK_INTERRUPTIBLE);
    spin_unlock_irq(&pool->lock);
    schedule();
    goto ↑woke_up;
} ? end worker_thread ?

```

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/time.h>
#include <linux/timer.h>
#include <linux/workqueue.h>
#include <asm/atomic.h>

```

```

struct timer_data {
    struct timer_list timer;
    unsigned long prev_jiffies;
    unsigned int loops;
};
struct timer_data test_data1;
struct timer_data test_data2;

```

```

static void do_work(struct work_struct *work)
static DECLARE_WORK(test_work1, do_work);
static DECLARE_WORK(test_work2, do_work);
static struct workqueue_struct *test_workqueue;

```

```

atomic_t wq_run_times;
static unsigned int failed_cnt = 0;

```

```

void test_timer_fn1(unsigned long arg)
{
    struct timer_data *data = (struct timer_data *)arg;
    unsigned long j = jiffies;
    data->timer.expires += 2 * HZ;
    data->prev_jiffies = j;
    add_timer(&data->timer);
    if (queue_work(test_workqueue, &test_work1) == 0) {
        printk("Timer (0) add work queue failed\n");
        failed_cnt++;
    }
    data->loops++;
    printk("timer-0 loops: %u\n", data->loops);
}

```

```

int wq_init(void){
    unsigned long j = jiffies;
    printk("jiffies: %lu\n", jiffies);
    atomic_set(&wq_run_times, 0);
    init_timer(&test_data1.timer);
    test_data1.loops = 0;
    test_data1.prev_jiffies = j;
    test_data1.timer.function = test_timer_fn1;
    test_data1.timer.data = (unsigned long)(&test_data1);
    test_data1.timer.expires = j + 2 * HZ;
    add_timer(&test_data1.timer);

    init_timer(&test_data2.timer);
    test_data2.loops = 0;
    test_data2.prev_jiffies = j;
    test_data2.timer.function = test_timer_fn2;
    test_data2.timer.data = (unsigned long)(&test_data2);
    test_data2.timer.expires = j + 3 * HZ;
    add_timer(&test_data2.timer);
    test_workqueue = create_singlethread_workqueue("test-wq");
    printk("test-wq init success\n");
    printk("jiffies: %lu\n", jiffies);
    return 0;
}

```

```

void test_timer_fn2(unsigned long arg)
{
    struct timer_data *data = (struct timer_data *)arg;
    unsigned long j = jiffies;
    data->timer.expires += 3 * HZ;
    data->prev_jiffies = j;
    add_timer(&data->timer);
    if (queue_work(test_workqueue, &test_work2) == 0) {
        printk("Timer (1) add work queue failed\n");
        failed_cnt++;
    }
    data->loops++;
    printk("timer-1 loops: %u\n", data->loops);
}

void do_work(struct work_struct *work)
{
    atomic_inc(&wq_run_times);
    printk("====work queue run times:
           %u====\n", atomic_read(&wq_run_times));
    printk("====failed count: %u====\n", failed_cnt);
}

```

```

void wq_exit(void)
{
    del_timer(&test_data1.timer);
    del_timer(&test_data2.timer);
    destroy_workqueue(test_workqueue);
    printk("wq exit success\n");
}

MODULE_AUTHOR("Xidian");
MODULE_LICENSE(" GPL");
module_init(wq_init);
module_exit(wq_exit)

```